Projection with Select()

Return only the data your application needs

1. Core idea

`Select()` creates a projection: it chooses which properties/columns the query returns. Instead of loading complete entities, you can return only the fields needed for a screen, report, or API response.

2. Why projection matters

Loading full entities	Using projection
May fetch every mapped column for each entity.	Fetches only the selected columns when the projection is translated to SQL.
Transfers more data than needed for simple displays.	Reduces data transfer and the amount of data materialized.
Useful when the application needs the entity itself.	Ideal for lists, dashboards, reports, and API DTOs.

Mental model: Entity query → Select(...) → only needed columns → less data transferred.

3. Short example

If a list displays only student names, project to those two properties instead of retrieving full Student entities.

StudentNames.cs

```
var students = context.Students
    .Select(s => new
    {
        s.FirstName,
        s.LastName
    })
    .ToList();
```

The result is a list of anonymous objects containing only FirstName and LastName.

4. What SQL does EF Core generate?

For the projection above, SQL typically selects only the requested columns, for example:

GeneratedSQL.sql

```
SELECT [s].[FirstName], [s].[LastName]
FROM [Students] AS [s];
```

Use `ToQueryString()` to preview the SQL query shape before execution. Runtime logging can show the command that is actually executed.

5. Common mistake

Avoid `context.Students.ToList()` when the screen only needs student names. That loads full entity data unnecessarily. Prefer a projection that selects only the required properties.

6. Practical notes

- Projection is commonly used in API responses, dashboards, reports, and list screens.
- `ToList()` executes/materializes the query; `Select()` shapes the result.
- Projection can return an anonymous type or a DTO, depending on what the application needs.

Mental anchor: select only what you need — but load full entities when the operation genuinely requires them.

Next lesson: combining Where + Select + OrderBy.